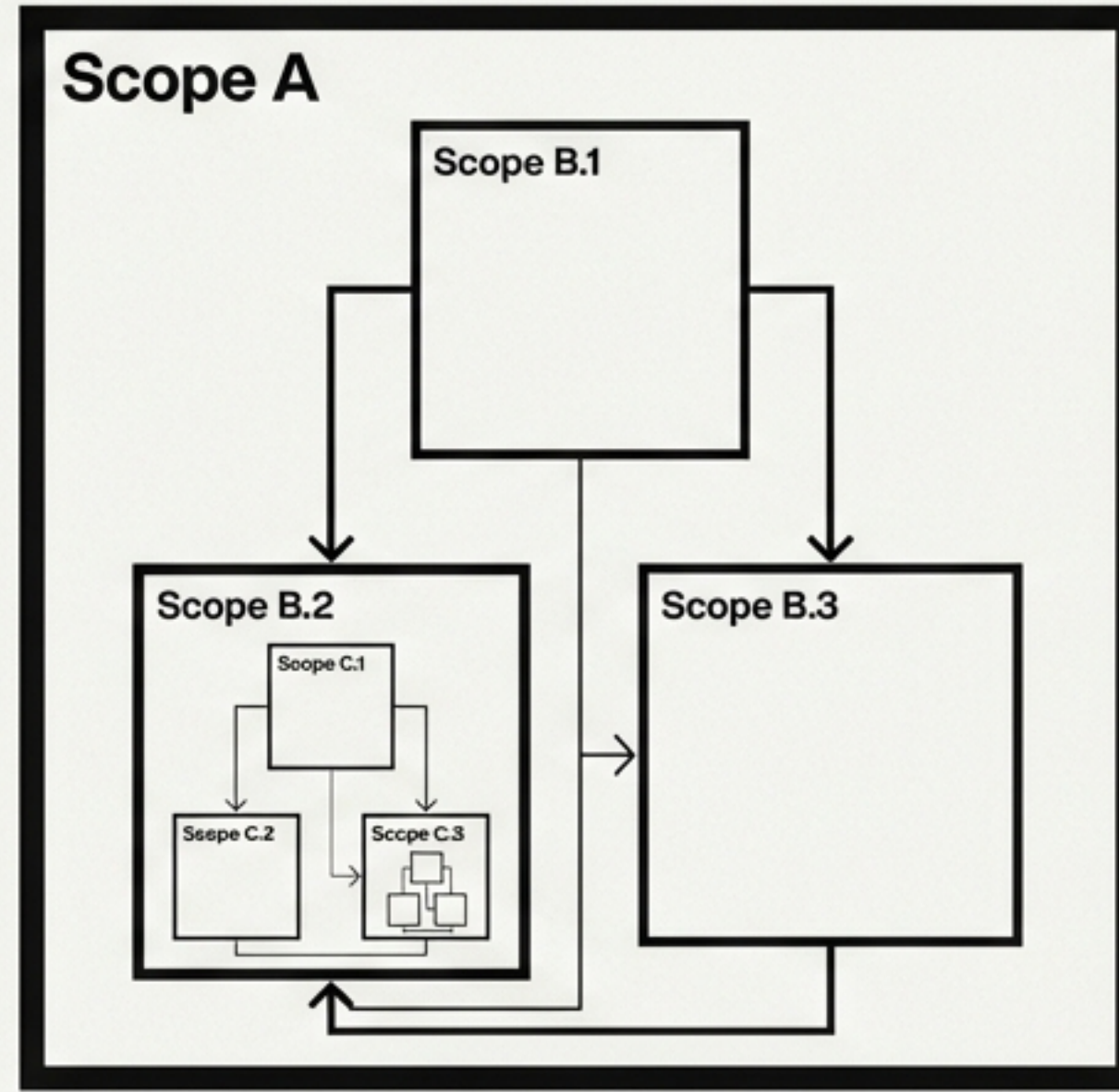


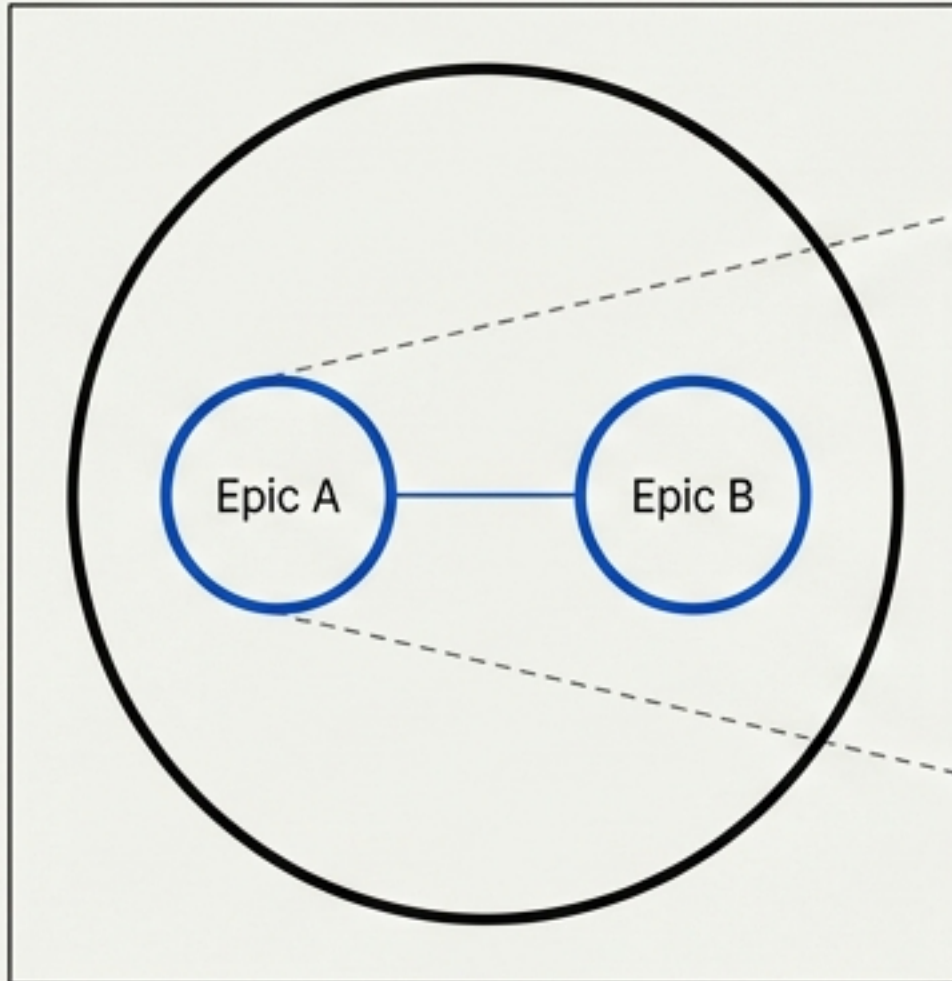
On Fractal Scopes

The Recursive Architecture of Issues-FS

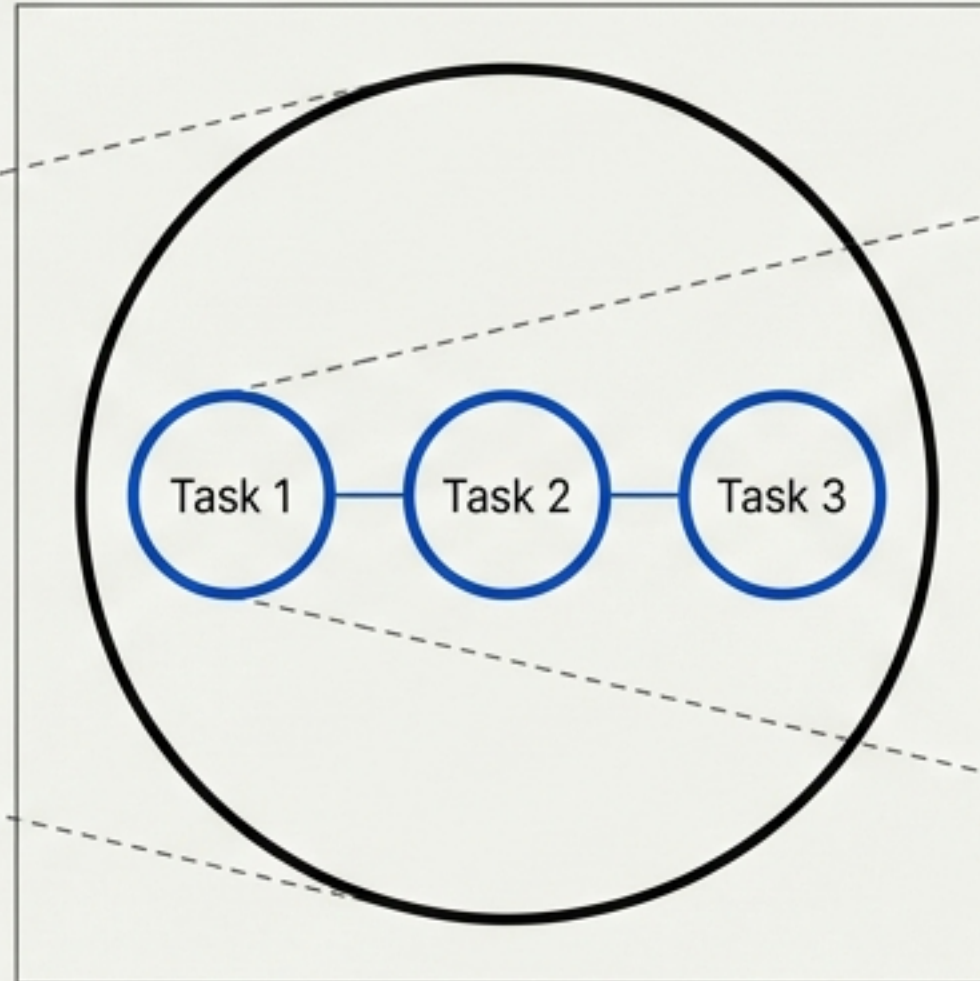


Graphs of Graphs of Graphs

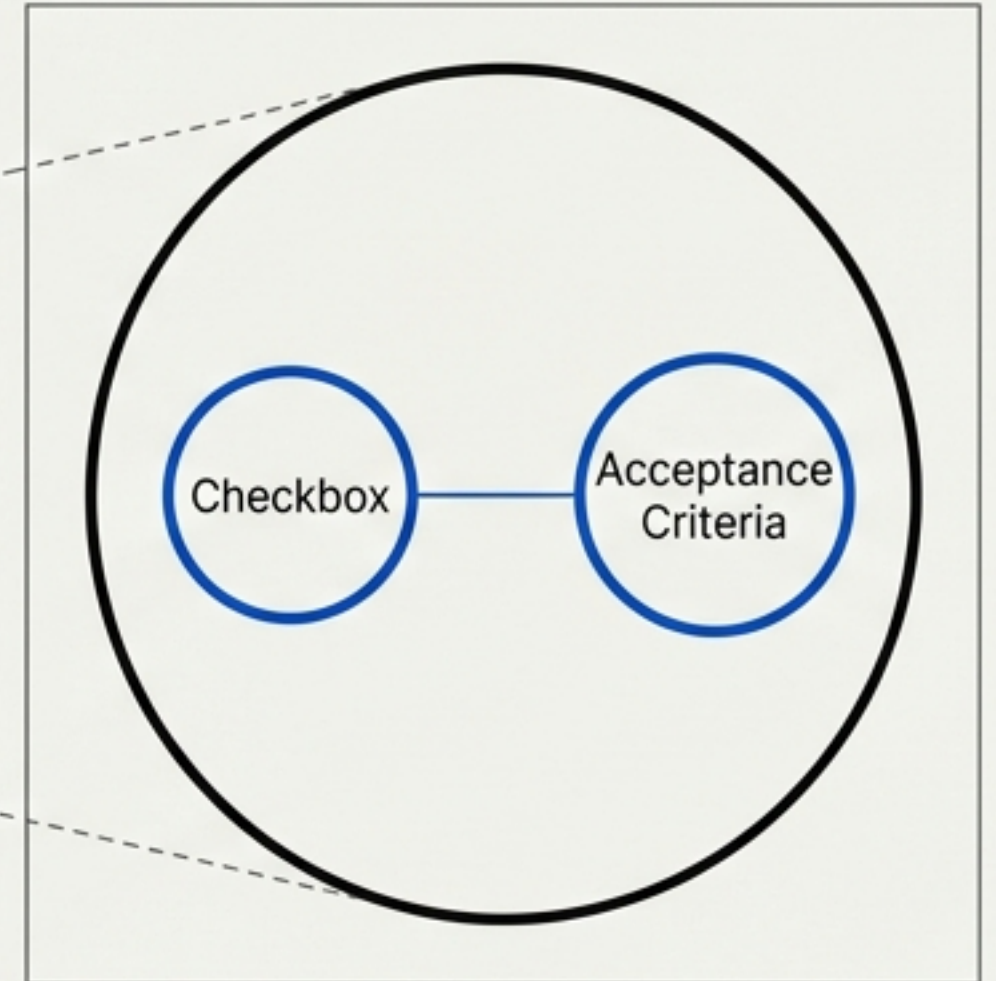
Project Scope



Epic Scope



Task Scope

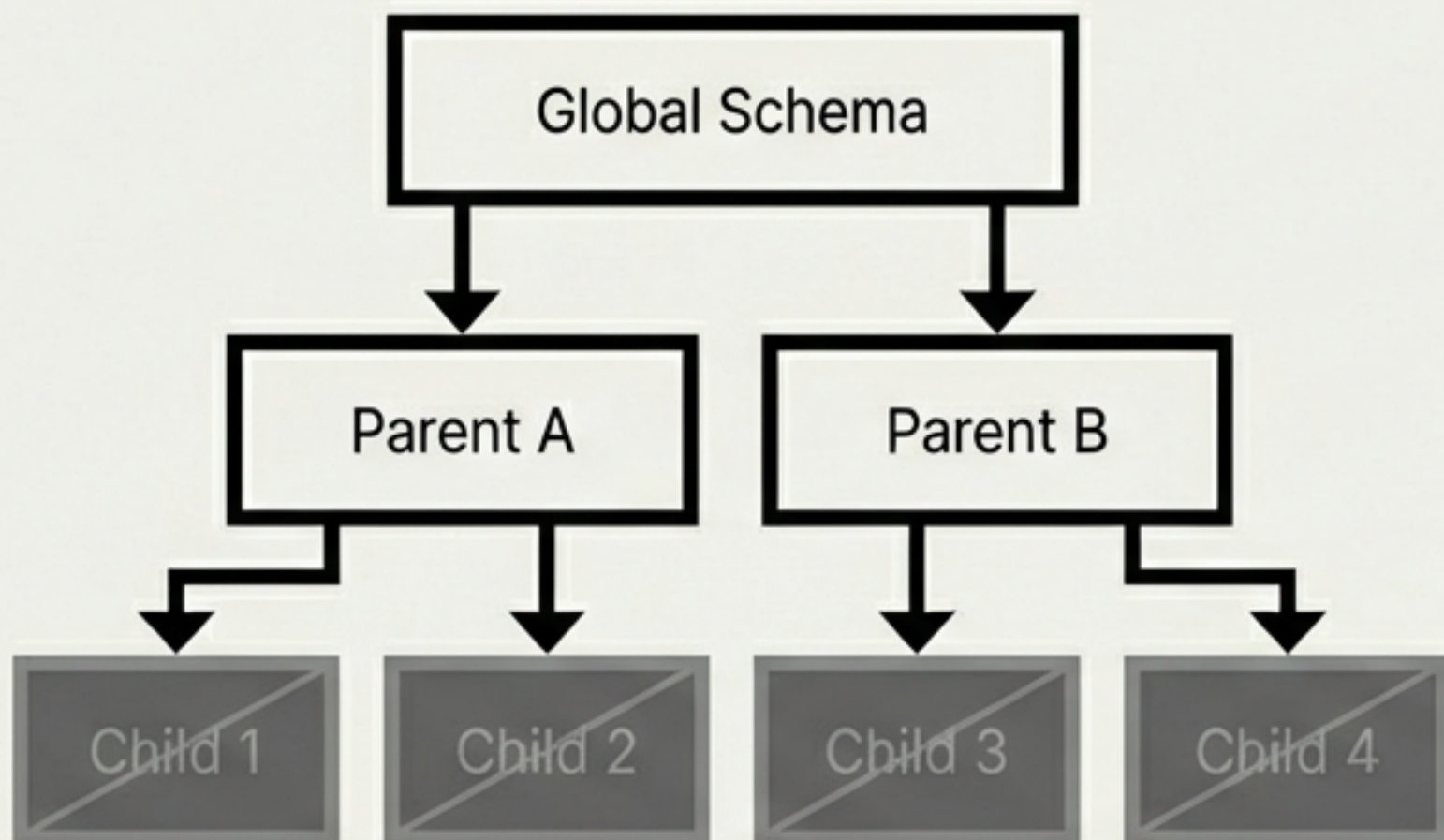


Issues-FS is not a single graph. It is a recursive structure where the same pattern—nodes, edges, connectivity—repeats identically at every level of zoom. Every level (Repo, Folder, Issue, Task, Checkbox) defines its own ontology and taxonomy.

"What we're doing now is a fractal way of capturing information... Every level—repo, folder, issue—can define its own ontology. That's the power of the fractal design."
— Dinis Cruz (Stakeholder Interview, 10 Feb 2026)

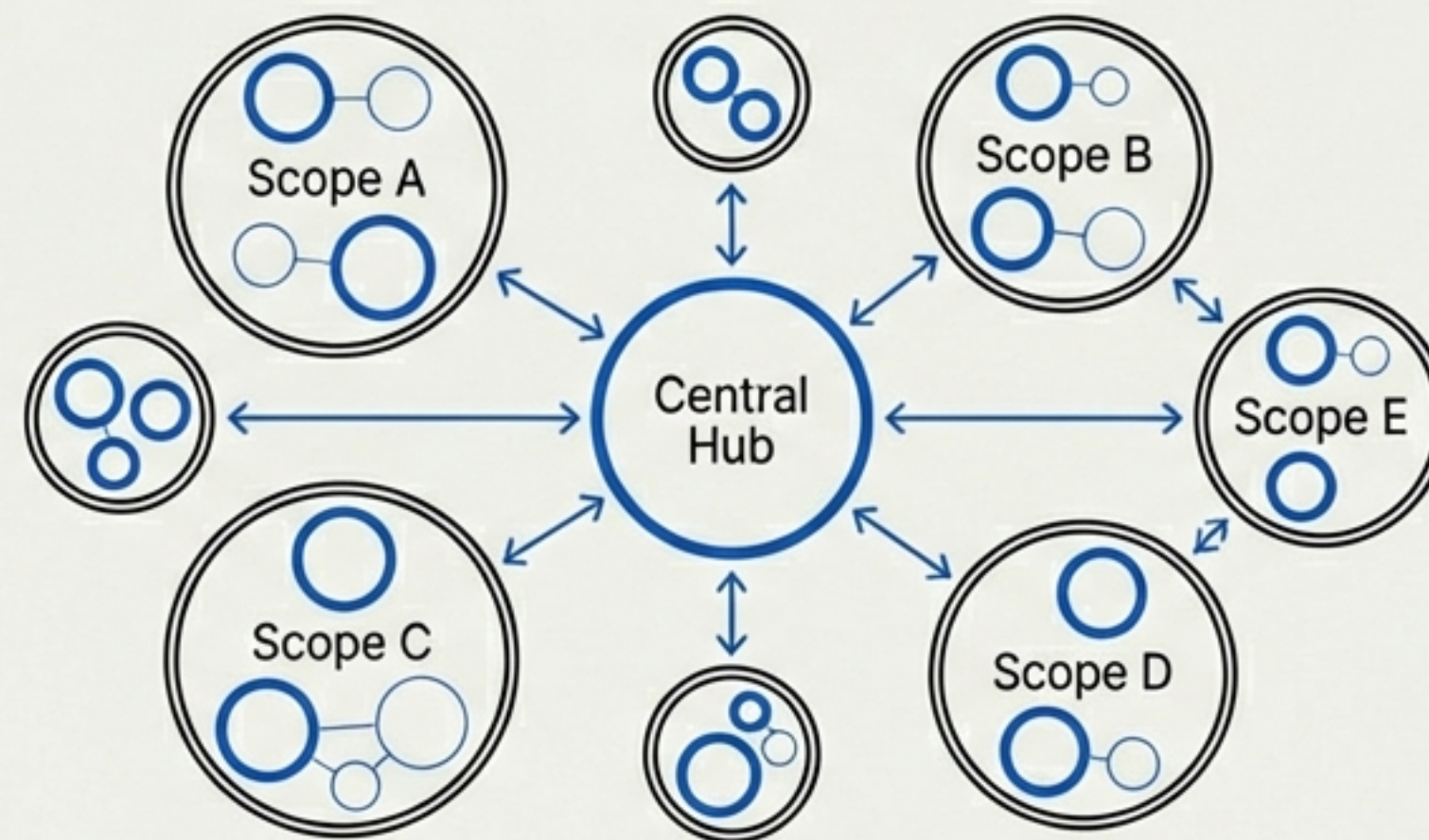
Moving From Enforcement to Autonomy

HIERARCHIES (Traditional)



- Top-down enforcement.
- Child elements inherit parent's schema.
- Goal: Conformity.
- Result: Rigid, fragile systems that reject reality.

FRACTALS (Issues-FS)

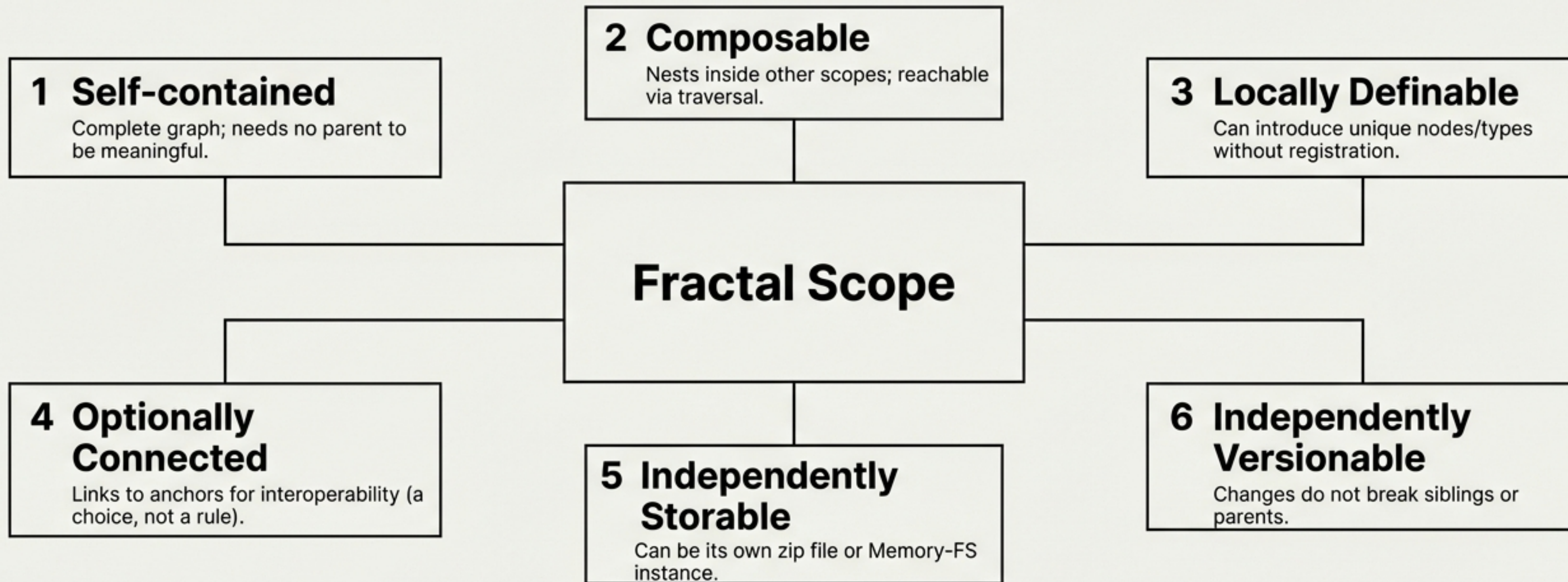


- Self-similar at every scale.
- Child scopes are autonomous; they do NOT inherit.
- Goal: Connectivity.
- Result: Scopes link to parents optionally for interoperability.

Takeaway: A fractal system surfaces differences rather than forcing a false consistency.

The Formal Pattern: pattern__fractal_scope

Base Class: Base__Scope.py

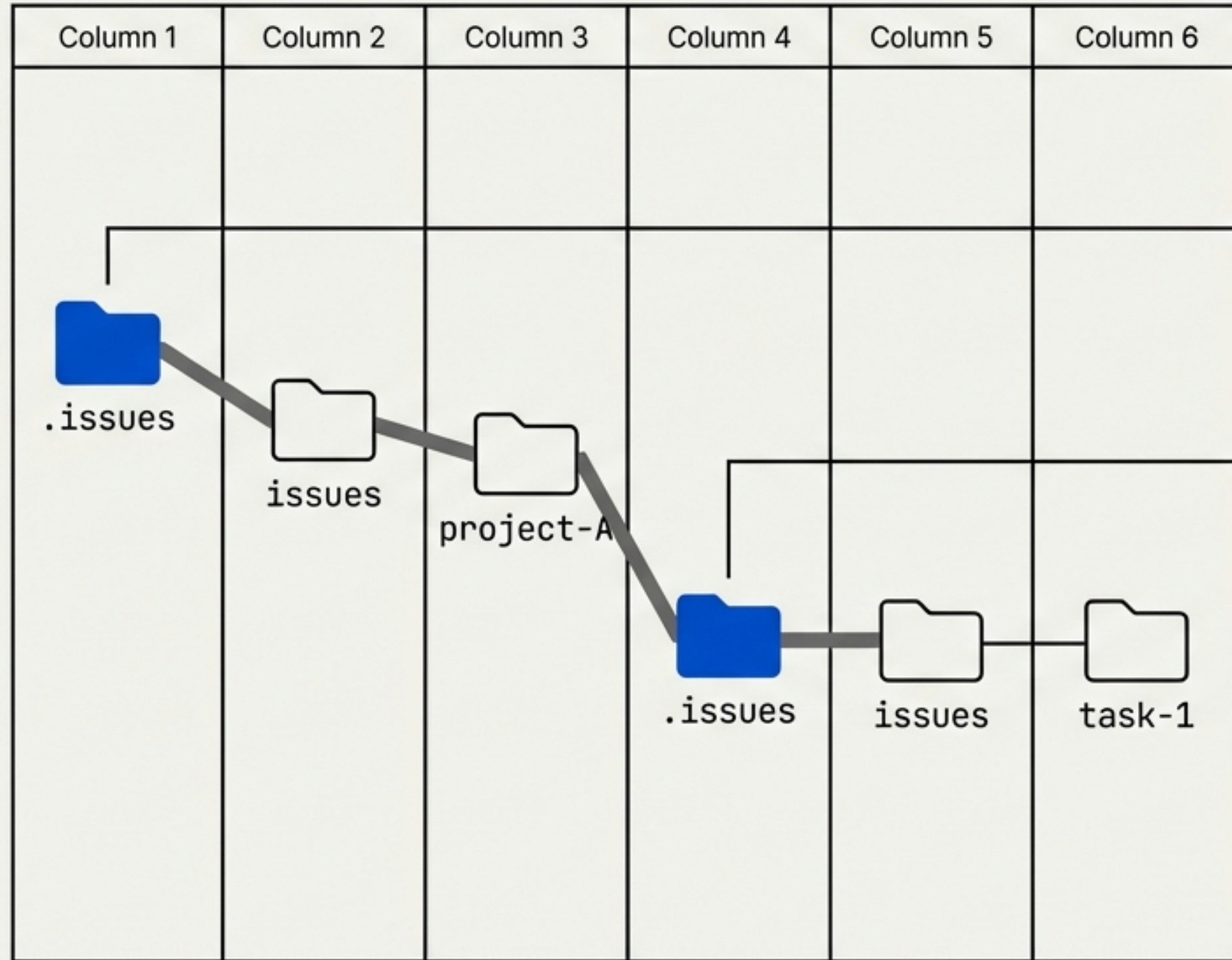


Definition: A thing that can define its own vocabulary.

Layer 1: The Filesystem

The Recursive Discovery

The system uses 'nodes_list_all()' to walk the tree. Unlike Git, which struggles with nested repos/submodules, Issues-FS treats nesting as a native property.



Each directory is a potential complete database.

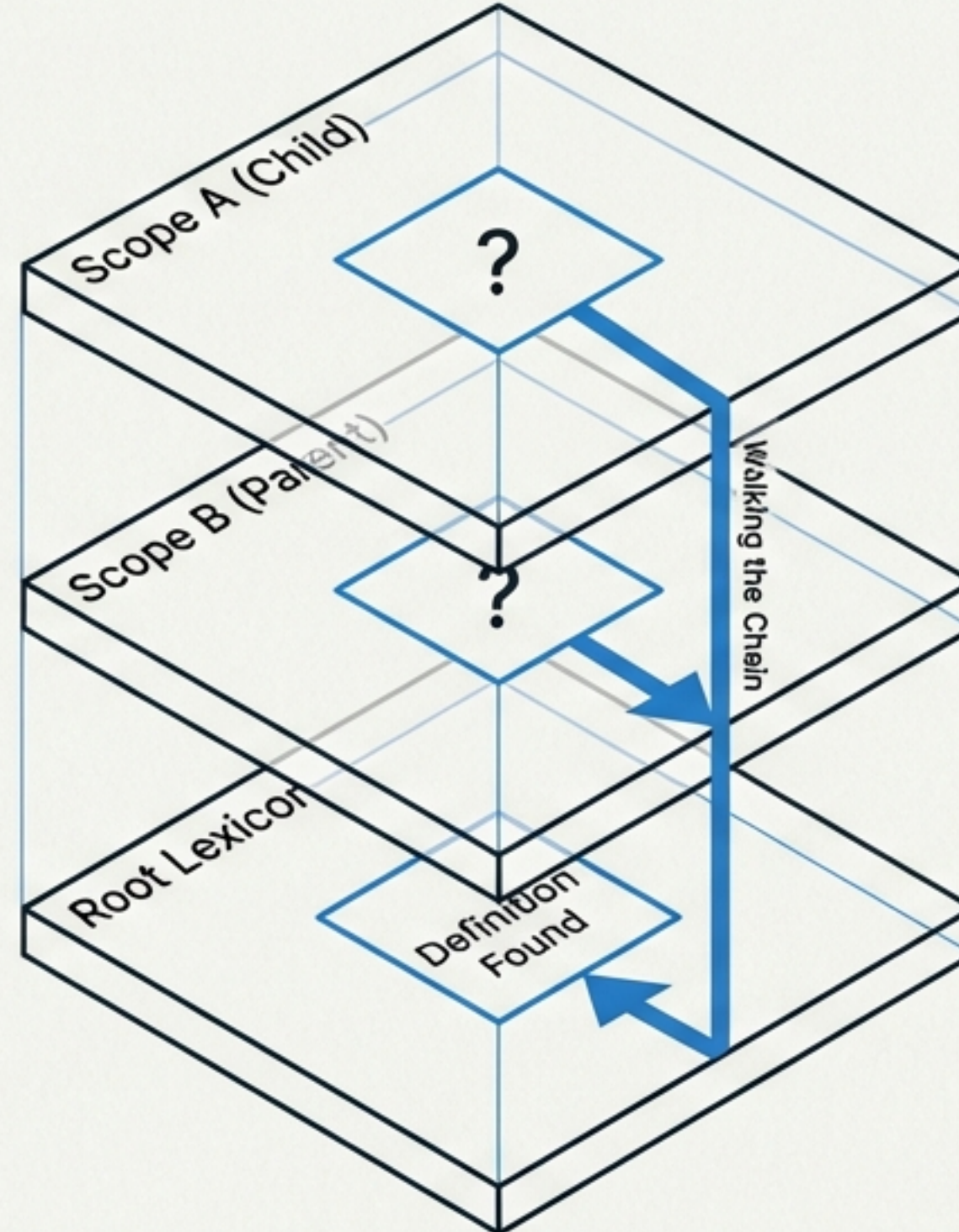
Differentiation

- Git: Submodules are bolted on.
- Issues-FS: Native nesting. Every directory handles 'config/node-types.json' independently.

Layer 2: Scope Resolution & Vocabulary

The Problem: If every scope is autonomous, how do we share meaning?

The Solution: If a definition isn't found locally, the system looks up. If not found at the root, it reports a gap.

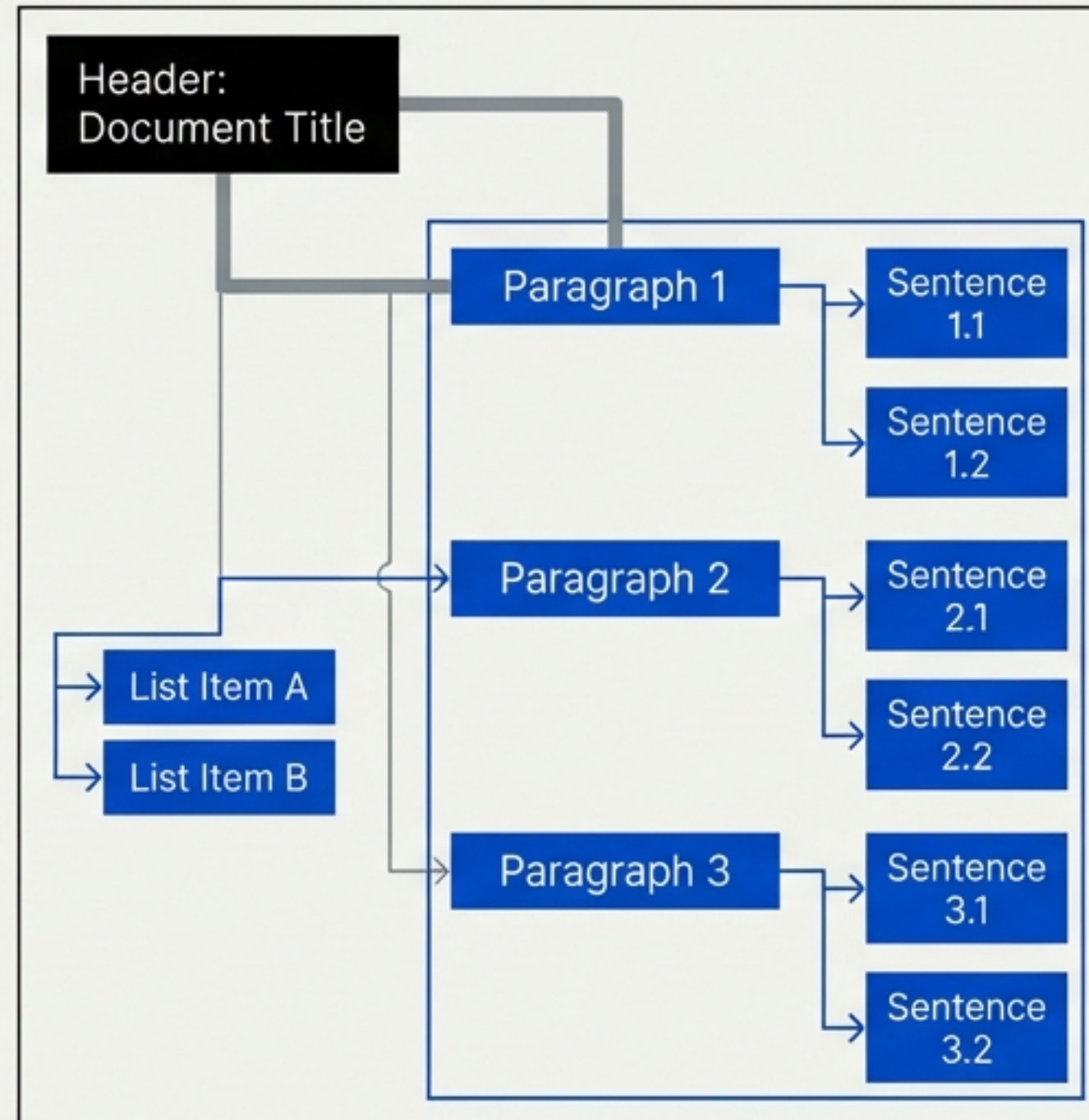


Reality Check

- Name Clashes are normal.
"Task-23" in Scope A !=
"Task-23" in Scope B.
- Divergence: Two projects can define "Review" differently.
Both are valid.

Layer 3: The Fractal Document

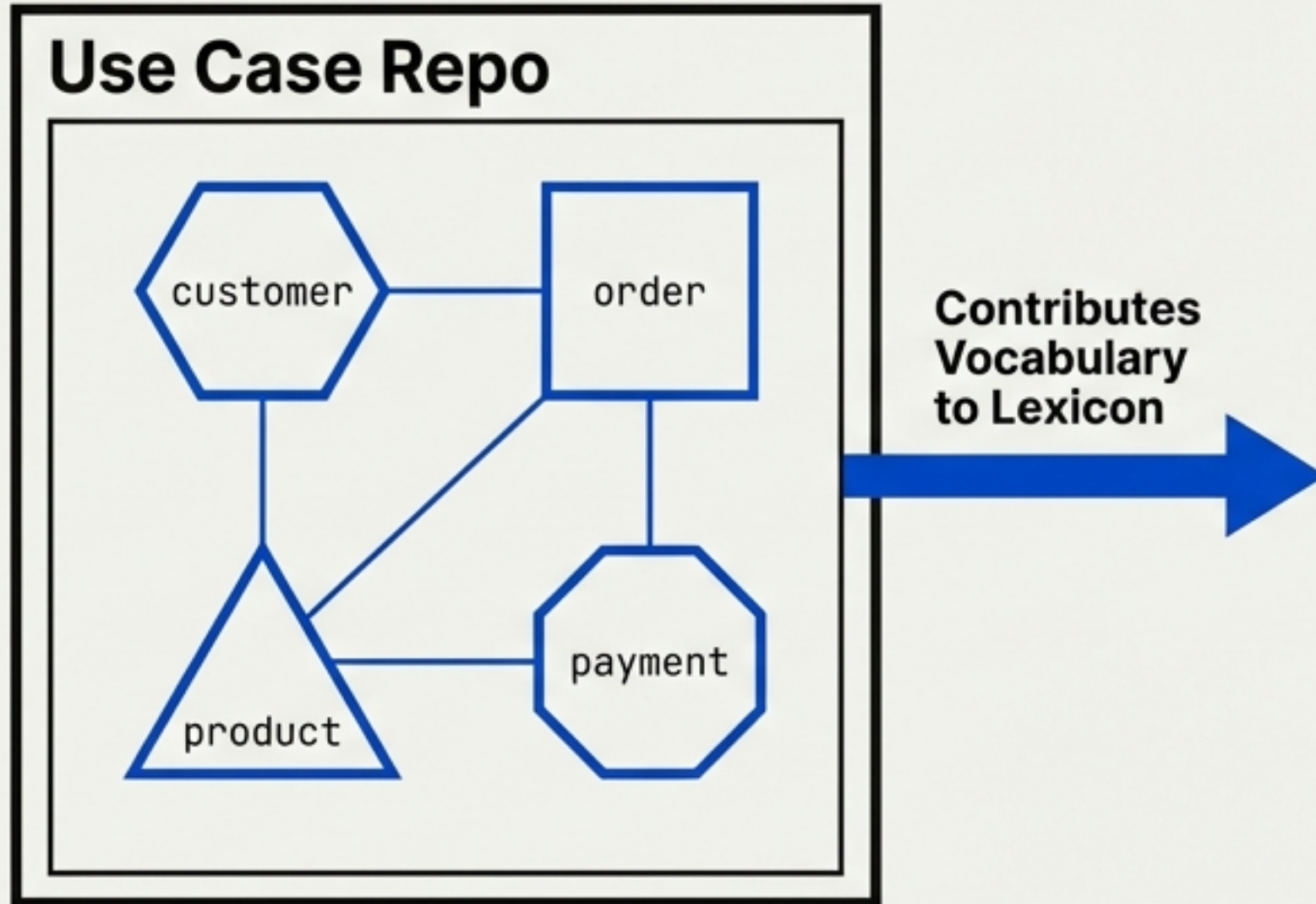
Core Assertion: A document IS a fractal scope.



- **Structure:** Documents are not flat text files; they are graphs of Sections and Paragraphs.
- **Independent Storage:** Load a specific paragraph without the whole document.
- **Composability:** Combine paragraph graphs from different sources.
- **Bidirectional Zoom:** Expand a sentence into a paragraph, or summarize a section into a node.

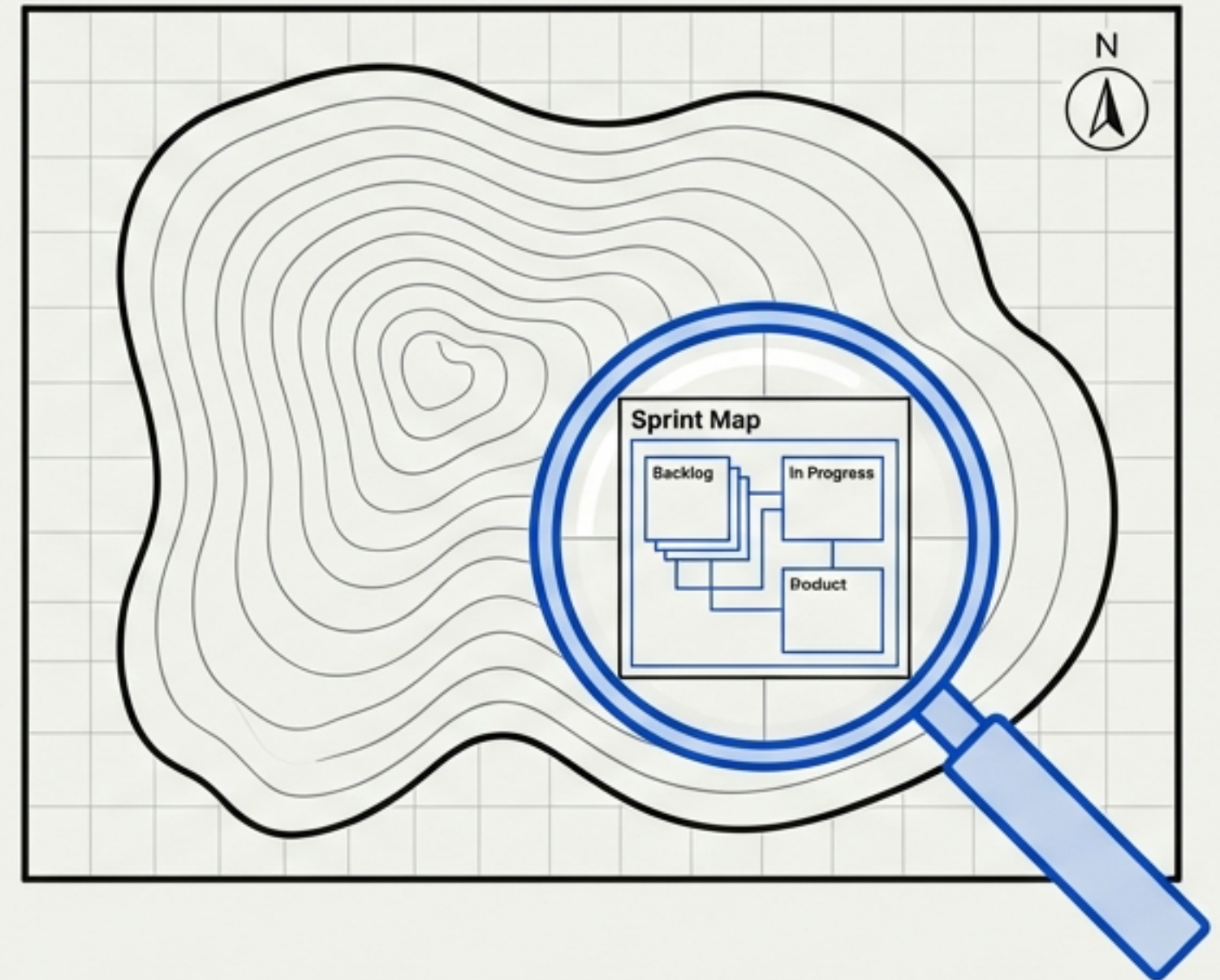
Layers 4 & 5: Use Cases and Maps

Use Cases as Participants



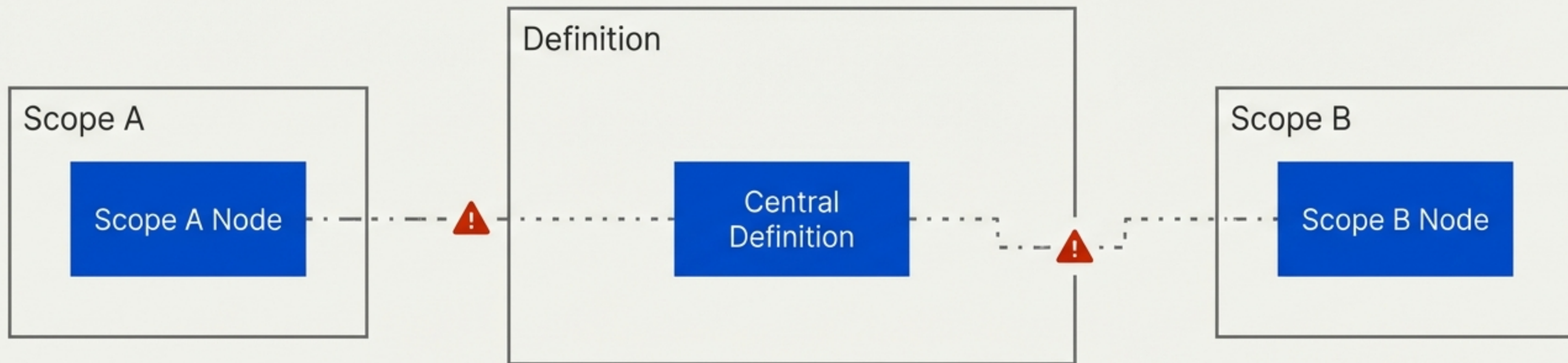
- Not external scripts, but full graph participants.
- Define domain-specific nodes.

The Cartographer's View



- Maps of maps of maps.
- A map of a 'Role' or a 'Sprint' is just another scope.

Embracing the Mess



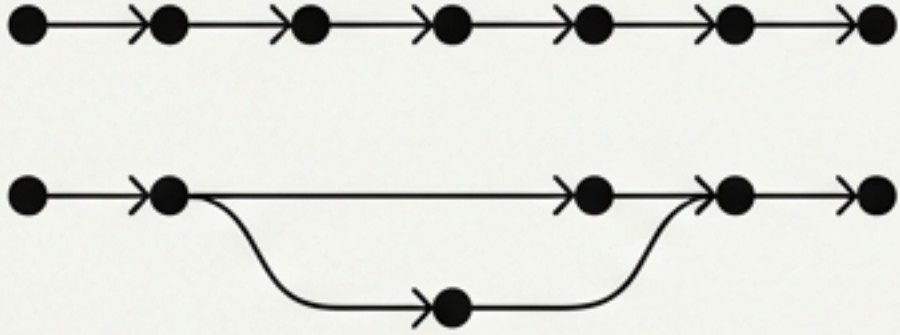
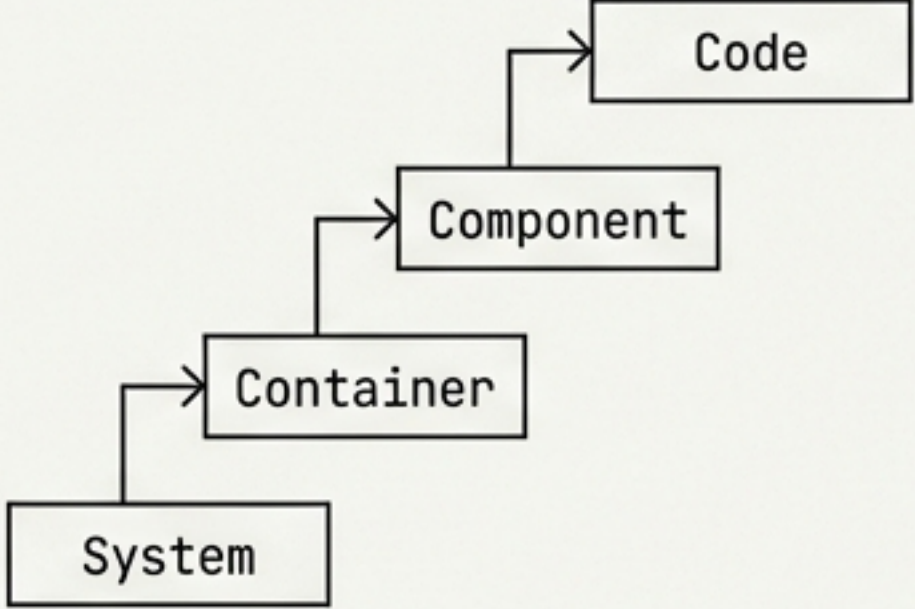
The Reality: A distributed system will inevitably have gaps and conflicts.

The System's Response: Treat them as **OBSERVATIONS**, not errors.

- ⚠ Gaps: "Type X is referenced but not defined."
- ⚠ Partial Mappings: "Matches 3 of 5 aspects."
- ⚠ Contradictions: "Scope A says Closed is terminal; Scope B says Reopened is allowed."

The system reports reality rather than forcing a false consistency.

Architectural Comparisons

vs. Git	vs. Semantic Web	vs. C4 Model
 Linear DAG (No Cycles)	<pre>id: node_001 type: Person name: 'Alice' knows: 'Bob' works_at: 'CorpX' role: 'Engineer' location: 'CityY' status: 'Active' since: 2023-01-01 attributes: -- responsible: {1} sehewa: "corpX" lecalocation: 'Romings' status: "None" ...</pre> Node IS Schema	
Git is linear/append-only. Issues-FS is a Graph (cycles allowed) with native nesting.	Semantic Web: Node IS Schema. Issues-FS: Meaning emerges from Edges (Node connected_to Schema).	C4 has fixed levels (System > Container > ...). Issues-FS has emergent levels; complexity scales infinitely.

Implementation Status

IMPLEMENTED (The Structure)

- ✓ - **Filesystem nesting**
(`issues/issues/...``)
- ✓ - **Recursive node discovery**
(`nodes_list_all`)
- ✓ - **Per-scope configuration**
(`node-types.json`)

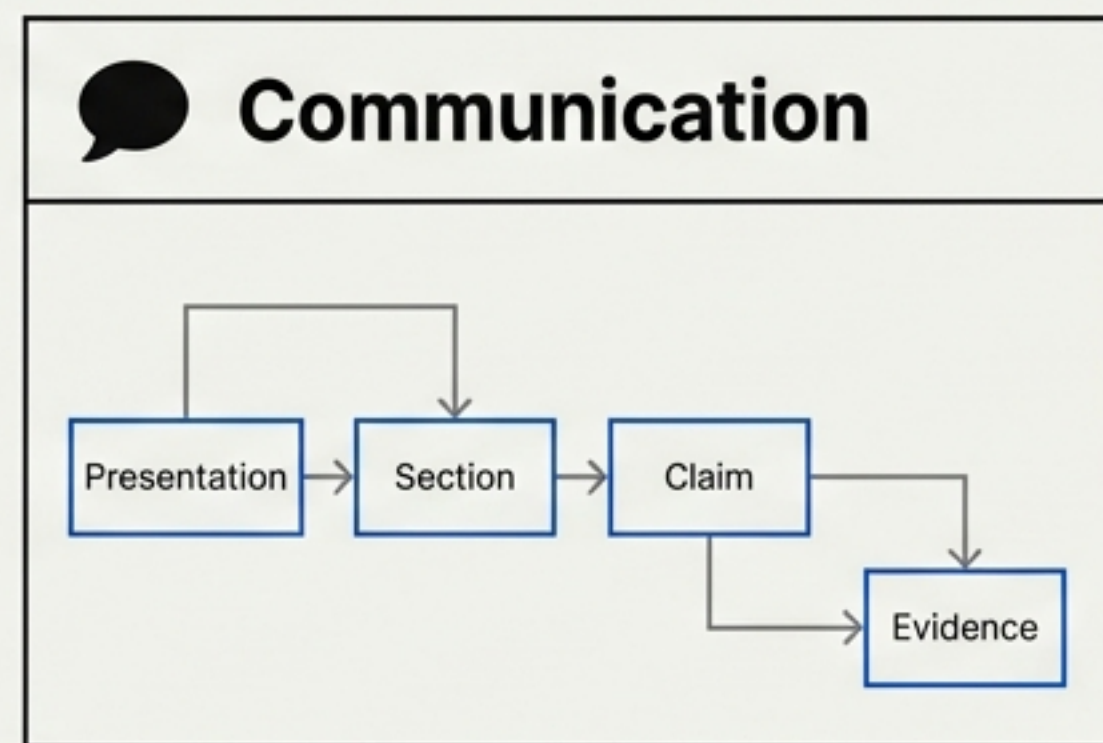
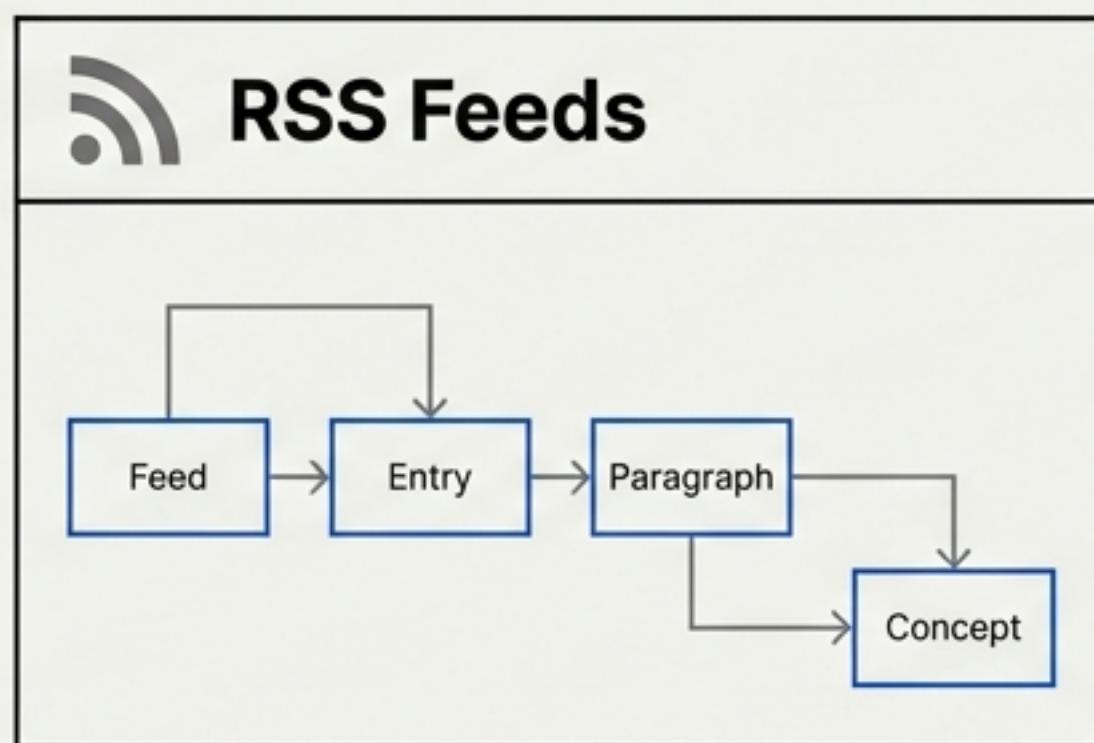
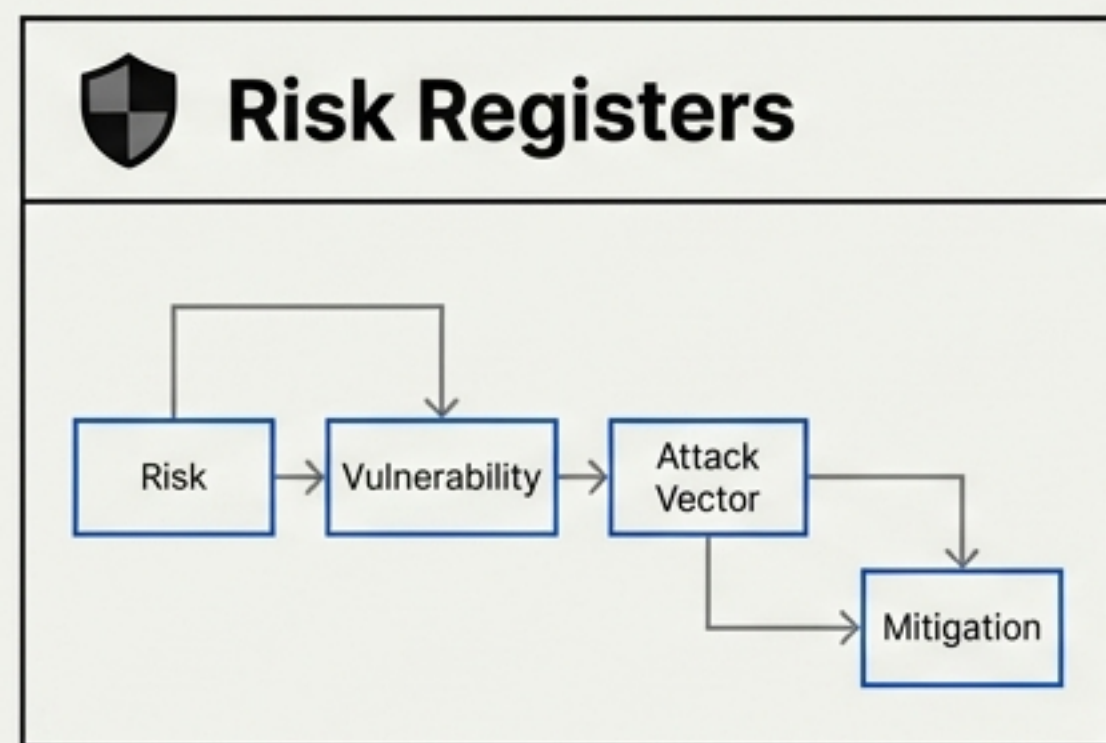
DESIGNING (The Intelligence)

- ✎ - **Scope resolution engine**
(walking the chain)
- ✎ - **Cross-graph edge support**
- ✎ - **Analysis tooling**
(conflict surfacing)

Note: The structure is mature; the tooling is catching up.

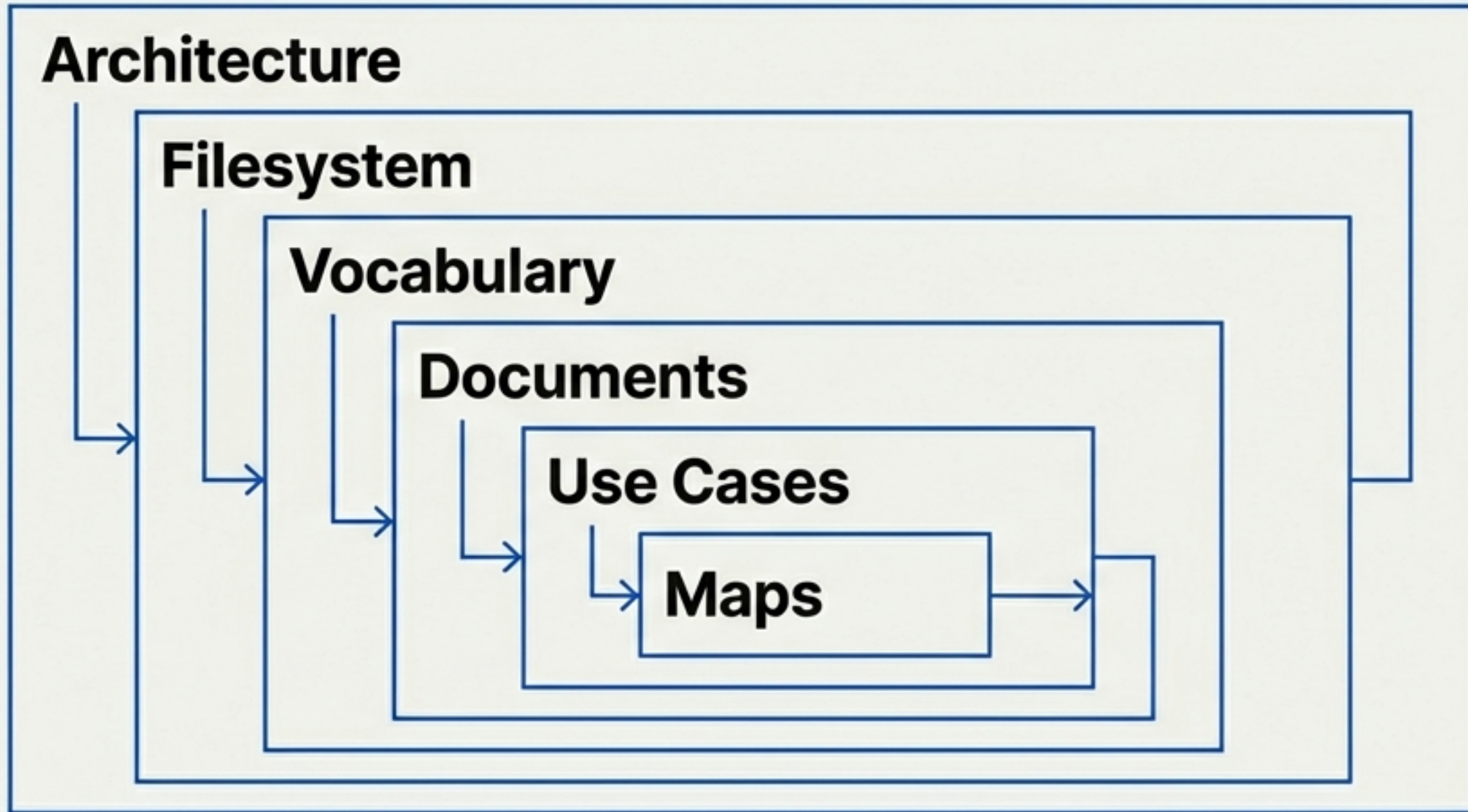
Beyond Issue Tracking

Modeling any complex domain using the same architecture.



The same structural pattern—nodes, edges, connectivity—applies everywhere.

The Code Must Reflect the Design



- **Fractal Principle:**
Not a feature, but the architecture.
- **5 Layers:**
From disk structure to user maps.
- **Philosophy:**
Honest observation over forced conformity.

We are building a system where complexity can scale **infinitely** without breaking the fundamental architecture.

Source Map

- Thinking in Graphs (Parts 3 & 6)
- Lexicon Architecture v2 (The Formal Model)
- Stakeholder Interview (10 Feb 2026 - Vision)
- Semantic Text Architecture (Fractal Documents)
- Briefs: File Structure Refactoring, Git Comparison
- Librarian Side Capture

Concept compiled by the Librarian from 35+ active source files.